

AWS CERTIFIED · STUDY NOTES

AWS Data Engineer Associate — DEA-C01 핵심 정리

도메인을 가로지르는 주제별 요약. 각 장은 판단 기준 → 서비스별 상세 → 시험 포인트
(합정) 순서로 정리했다.

34%

데이터 수집·변환

26%

데이터 스토어 관리

22%

데이터 운영·지원

18%

데이터 보안·거버넌스

65문항 · 130분 · 720/1000 합격 | 인채용 스냅샷 · 2026-09-03 | 원본 글이 항상 최신

목차

1. 데이터 수집 (Ingestion)	4
1.1 먼저 판단할 것 — 스트리밍 vs 배치 vs CDC	4
1.2 스트리밍 수집 서비스 비교	5
1.3 Kinesis Data Streams 상세	5
1.4 Kinesis Data Firehose 상세	6
1.5 Amazon MSK 상세	6
1.6 배치 수집 · CDC	6
2. 데이터 변환 · 처리 (Transform)	7
2.1 변환 서비스 선택 기준	7
2.2 AWS Glue 상세	7
2.3 Amazon EMR 상세	8
2.4 Spark 튜닝 포인트	8
3. 파이프라인 오케스트레이션	9
3.1 도구 선택	9
3.2 Step Functions 세부	9
3.3 패턴	9
4. 파일 포맷 · 압축 · 파티셔닝 (성능·비용의 핵심)	10
4.1 포맷 비교	10
4.2 압축 — splittable 여부가 핵심	10
4.3 비용 절감 공식 (Athena/Spectrum)	10
4.4 파티셔닝	11
4.5 작은 파일 문제	11
4.6 트랜잭션 테이블 포맷 (레이크하우스)	11
5. 데이터 스토어 선택 · S3 수명주기	11
5.1 요구사항 → 스토어	11
5.2 Amazon S3	12
6. Amazon Redshift 심화	12
6.1 아키텍처·노드	12

6.2 분산 스타일(DISTSTYLE)	13
6.3 정렬 키(SORTKEY)	13
6.4 적재/추출	13
6.5 워크로드 관리	13
6.6 레이크·외부 연동	13
6.7 유지관리·최신 기능	13
6.8 보안·관측	14
7. Amazon DynamoDB 심화	14
7.1 키 설계	14
7.2 용량 모드	14
7.3 핫 파티션·스로틀링 대응	14
7.4 인덱스	14
7.5 부가 기능	15
7.6 데이터 모델링	15
8. Athena · Glue Data Catalog · Lake Formation	15
8.1 Amazon Athena	15
8.2 AWS Glue Data Catalog	16
8.3 AWS Lake Formation	16
9. 데이터 모델링 · 스키마 진화 · 리니지	16
10. 운영 · 모니터링 · 로깅	17
11. 데이터 품질	17
12. 데이터 보안 (인증 · 인가 · 암호화)	18
12.1 IAM	18
12.2 인증 방식	18
12.3 시크릿 관리	18
12.4 저장 데이터 암호화 (S3)	18
12.5 전송 중 암호화	19
12.6 KMS	19
12.7 네트워크 격리	19
12.8 인가(권한)	19

13. 데이터 거버넌스 · 프라이버시 · 감사	19
14. 자주 나오는 시나리오 · 암기 포인트	20
수집	20
변환·처리	20
오케스트레이션	20
포맷·레이크	21
스토어	21
Redshift	21
DynamoDB	21
보안·거버넌스	21
운영·품질	21
15. 마지막 점검 — 헛갈리는 짝 비교	22

AWS Certified Data Engineer - Associate(DEA-C01) 대비 핵심 개념 정리다. 특정 서비스 매뉴얼이 아니라 **도메인을 가로지르는 주제별 요약**이라, 공부하면서 계속 고쳐 나갈 예정이고 이 글이 원본이다. 인쇄용 스냅샷은 아래 PDF로 받으면 된다.

시험 도메인 비중 — ① 데이터 수집·변환 34% · ② 데이터 스토어 관리 26% · ③ 데이터 운영·지원 22% · ④ 데이터 보안·거버넌스 18%. 65문항 / 130분 / 1000점 만점에 720점 합격(스케일 점수). 문항은 "요구사항 → 가장 적합한 서비스·설정" 형태가 대부분이라, 서비스별 **결정 기준과 함정**을 외우는 게 핵심이다.

⚠ 학습용 요약본이다. 서비스 한도·기능은 AWS가 수시로 갱신하므로, 응시 전에는 반드시 공식 문서와 최신 Exam Guide를 확인하자.

이 문서를 읽는 법 — 각 장은 ① **판단 기준(무엇을 언제)** → ② **서비스별 상세** → ③ **시험 포인트(함정)** 순서로 정리했다. 시간이 없으면 각 장의 표와 "시험 포인트"만 봐도 된다.

1. 데이터 수집 (Ingestion)

1.1 먼저 판단할 것 — 스트리밍 vs 배치 vs CDC

신호(요구사항 키워드)	선택
"실시간", "수 초 이내", "이벤트가 계속 흘러들어온다", "여러 소비자"	스트리밍 (Kinesis / MSK)
"매시간", "매일 밤", "파일이 떨어지면", "벌크 로드"	배치 (S3 + 이벤트, Glue, DMS full load)
"원본 DB 변경을 계속 반영", "CDC", "복제 지연 최소화"	CDC (DMS, DynamoDB Streams, Zero-ETL)

신호(요구사항 키워드)	선택
"SaaS(Salesforce·SAP 등)에서 데이터를 가져온다"	AppFlow
"온프레미스 파일 서버·NFS·SMB를 통째로 옮긴다"	DataSync (지속 동기화) / Transfer Family(SFTP 인터페이스)
"페타바이트, 네트워크로는 무리"	Snow Family (Snowball Edge)

1.2 스트리밍 수집 서비스 비교

서비스	핵심 특성	쓰는 상황
Kinesis Data Streams (KDS)	샤드 기반. 샤드당 쓰기 1 MB/s·1,000 rec/s, 읽기 2 MB/s(공유) / Enhanced Fan-Out은 소비자별 2 MB/s. 보존 24h~365일. 파티션 키 단위 순서 보장. 다중 소비자, 재처리(replay) 가능. On-demand 모드 존재.	1초 이내 실시간, 커스텀 처리 로직, 여러 독립 소비자, 재생 필요
Kinesis Data Firehose	완전관리형 서버리스 적재(delivery) . 버퍼(1~128 MB / 60~900초) 후 전송. 대상: S3, Redshift, OpenSearch, Splunk, 서드파티 HTTP. Lambda로 변환, JSON→Parquet/ORC 변환, 동적 파티셔닝. 최소 지연 약 60초. 보존·재생 없음 .	코드 최소화 S3/DW에 근실시간 적재, 포맷 변환·압축
Amazon MSK	완전관리형 Apache Kafka. 파티션·컨슈머 그룹, 보존 자유. MSK Connect(커넥터), MSK Serverless. Glue 스키마 레지스트리 연동.	이미 Kafka를 쓰거나 Kafka 생태계(커넥터·정확히 1회 등)가 필요
DynamoDB Streams	테이블 항목 변경 캡처(보존 24h). Lambda 트리거 또는 Kinesis Data Streams for DynamoDB.	DynamoDB CDC, 변경 이벤트 기반 처리
Amazon SQS	메시지 큐. 소비 후 삭제, 단일 논리 소비자(팬아웃 아님). Standard(최소 1회, 순서 미보장) / FIFO(정확히 1회, 순서). DLQ.	작업 분산·디커플링(스트림 분석용이 아님)



KDS vs Firehose — 가장 자주 나오는 비교. "변환·적재만, 코드 없이, S3/Redshift로" → **Firehose**. "1초 미만 지연 / 여러 소비자가 같은 데이터를 각자 처리 / 나중에 다시 읽기" → **KDS**. Firehose는 최소 버퍼 때문에 **60초 미만 지연이 불가능**하다.

1.3 Kinesis Data Streams 상세

- 용량 계산** — 필요한 샤드 수 = `ceil( max(수집MB/s ÷ 1, 레코드/s ÷ 1000) )`, 읽기가 병목이면 소비자 수까지 고려. 초과 시 `ProvisionedThroughputExceededException` 발생.
- 처리량 초과 대응** — ① 샤드 추가(**re-sharding: split**), ② 파티션 키를 고엔트로피로 설계해 고르게 분산, ③ 프로듀서에서 지수 백오프 재시도, ④ 아예 **On-demand 모드**(자동 스케일, 관리 부담 제거, 비용은 처리량 비례).
- 소비 지연(컨슈머 lag) 감지** — `GetRecords.IteratorAge(Milliseconds)` 지표가 계속 커지면 소비자가 뒤처지는 중. 대응: 소비자 인스턴스/람다 병렬성 확대, **Enhanced Fan-Out(EFO)** 로 소비자별 전용 2 MB/s 확보, 처리 로직 최적화.
- 순서 보장** — 같은 파티션 키 → 같은 샤드 → 그 안에서 순서 보장. 리샤딩하면 부모 샤드를 완전히 소비한 뒤 자식 샤드를 읽어야 순서가 유지된다(KCL이 자동 처리).
- 프로듀서** — KPL(집계·배치로 처리량↑, 약간의 지연↑), Kinesis Agent(로그 파일 tail), SDK `PutRecord` / `PutRecords` (배치). **컨슈머** — KCL(체크포인트를 DynamoDB 테이블에 저장 → 중복 처리에 대비한 멍등성 필요), Lambda(이벤트 소스 매핑), Firehose, Managed Service for Apache Flink.

- **Lambda 소비 튜닝** — 배치 크기, 배치 윈도우, **병렬화 계수(샤드당 최대 10)**, 실패 시 `bisect on function error` (반씩 쪼개 문제 레코드 격리), DLQ(SQS/SNS), 최대 재시도·최대 레코드 수명.
- **암호화** — 저장 시 KMS(SSE), 전송 시 HTTPS. VPC에서 접근하려면 인터페이스 엔드포인트.

1.4 Kinesis Data Firehose 상세

- **버퍼링** — 크기(1~128 MB) 또는 시간(60~900초) 중 먼저 도달하는 조건으로 flush. 지연을 줄이려면 버퍼를 작게, 작은 파일 문제를 줄이려면 버퍼를 크게.
- **변환** — Lambda로 레코드 변환·필터·보강. 실패한 레코드는 S3의 `processing-failed/` 프리픽스로 따로 적재.
- **포맷 변환** — JSON → **Parquet/ORC**(Glue 카탈로그 스키마 필요). 압축(GZIP/Snappy/ZIP) 지원.
- **동적 파티셔닝** — 레코드 내용(예: `customer_id`, 날짜)으로 S3 프리픽스를 동적으로 만들어 적재 → Athena 파티션 프루닝에 유리. jq 표현식 또는 Lambda로 파티션 키 추출.
- **에러 처리** — 전송 실패 시 S3 백업 버킷(`errors/`)으로, Redshift 대상이면 중간 S3를 거쳐 COPY.

1.5 Amazon MSK 상세

- **구성** — 브로커(AZ 분산), 파티션·복제 계수(내구성), 컨슈머 그룹. **MSK Serverless**는 용량 계획 불필요(처리량 기반 과금), **프로비저닝**은 브로커 타입·수를 직접 선택.
- **MSK Connect** — Kafka Connect 관리형. S3 싱크, 디비지움(CDC) 소스 등 커넥터 실행.
- **인증/인가** — IAM 액세스 컨트롤, SASL/SCRAM(Secrets Manager), mTLS. ACL로 토픽 권한.
- **스키마** — AWS Glue Schema Registry로 Avro/JSON/Protobuf 스키마 호환성 강제(프로듀서/컨슈머 계약).
- **KDS와 차이** — MSK는 Kafka API·생태계·세밀한 보존/파티션 제어가 필요할 때. 순수 AWS 네이티브·완전 서버리스·운영 최소화는 KDS(특히 On-demand).

1.6 배치 수집 · CDC

- **AWS DMS** — 동종/이기종 DB 마이그레이션 + **CDC(지속 복제)**. 흐름은 `Full load → CDC`. 대상으로 S3·Kinesis·Redshift·OpenSearch·Kafka 가능 → **데이터 레이크 CDC 피드**. DMS Serverless(용량 자동), 그 외에는 복제 인스턴스 필요. 대용량 초기 적재는 병렬 로드·LOB 설정 주의.
- **AWS SCT (Schema Conversion Tool)** — 스키마/코드(뷰·프로시저) 변환. Oracle→Aurora PostgreSQL 등 **이기종일** 때만 필요(동종이면 불필요). 변환 난이도 평가 리포트 제공, 변환 불가 항목은 수동. 대용량은 SCT data extraction agent.
- **AWS AppFlow** — SaaS(Salesforce, SAP, Zendesk, Google Analytics...) ↔ AWS 양방향 전송. 필터·매핑·마스킹·검증 내장, 스케줄/이벤트 트리거. 코드 없이 SaaS 데이터를 S3/Redshift로.
- **AWS Transfer Family** — 관리형 SFTP/FTPS/FTP/AS2 → S3·EFS. 외부 파트너가 **SFTP로 올리는 인터페이스**가 필요할 때.
- **AWS DataSync** — 온프레미스 NFS/SMB/HDFS/오브젝트 ↔ AWS(S3/EFS/FSx) 대량·증분 전송, 스케줄·검증·암호화 내장. Transfer Family가 "프로토콜 엔드포인트 제공"이라면 DataSync는 "대량 복사·동기화 엔진".
- **S3 이벤트 트리거** — S3 Event Notifications → SNS/SQS/Lambda/EventBridge. **EventBridge** 경로가 필터·재시도·다중 타겟·아카이브 리플레이에 유리(권장).
- **Redshift 적재 경로** — `COPY` (대량 병렬, 기본), Auto-copy from S3(S3 이벤트로 자동 COPY), Streaming ingestion(Kinesis/MSK를 물리 뷰로 직접), **Zero-ETL**(Aurora→Redshift 무파이프라인 복제).

⚠ **시험 포인트** — ① "코드/서버 없이 스트림을 포맷 변환해 S3로" = Firehose. ② "온프레미스 DB를 최소 다운타임으로, 이후 변경분도 계속" = DMS(full load + CDC). ③ "Oracle → PostgreSQL 스키마" = SCT. ④ "S3에 파일 도착 → 파이프라인 시작" = S3 Event Notification → EventBridge → Step Functions/Glue. ⑤ Firehose는 재생 불가, KDS·MSK는 재생 가능.

2. 데이터 변환 · 처리 (Transform)

2.1 변환 서비스 선택 기준

서비스	특징 / 선택 기준
AWS Glue (Spark ETL)	서버리스 Spark. 인프라 관리 없음. 크롤러·카탈로그·스케줄·DQ 통합. 소~중 규모 ETL, 카탈로그 기반 레이크에 최적. 콜드스타트 약 1분.
Amazon EMR	관리형 Hadoop/Spark 클러스터. 대규모, 세밀한 튜닝, Hive/Presto/HBase/Flink/Hudi/Iceberg, Spot으로 비용 절감. 장기 클러스터·복잡 워크로드·기존 빅데이터 자산 이전.
AWS Lambda	경량·짧은(≤15분) 이벤트 변환. 파일 도착 시 소규모 처리, 라우팅, 간단 보강. 대용량·조인·셔플에는 부적합.
Amazon Redshift (ELT)	먼저 적재한 뒤 SQL로 변환(저장 프로시저, <code>MERGE</code>). 정형 DW 변환, Spectrum으로 S3까지 조인.
AWS Glue DataBrew	코드 없는 250+ 변환·정제·프로파일링. 분석가용, 재사용 레시피, 시각적.
Managed Service for Apache Flink	스트리밍 SQL/Java/Python. 윈도우 집계·이상탐지·조인. 실시간 스트림 분석.

판단 규칙: 서버리스 + 카탈로그 + 중간 규모 → Glue. 초대형 / 특수 엔진(HBase, Presto) / Spot 절감 / 오래 도는 클러스터 → EMR. 한 파일·수 초 처리 → Lambda. 데이터가 이미 DW에 있고 SQL로 충분 → Redshift ELT. 코드 없이 분석가가 직접 → DataBrew. 무한 스트림 집계 → Flink.

2.2 AWS Glue 상세

- **잡 유형** — Spark(배치), Spark Streaming(Kinesis/MSK 마이크로배치), **Python Shell**(경량, pandas·단일 노드), **Ray**(분산 파이썬).
- **워커 타입** — G.1X(4 vCPU/16 GB), G.2X, G.4X, G.8X(메모리 큰 조인·스큐), G.025X(스트리밍 저비용). 과금은 **DPU-시간**(초당, 최소 1분). Auto Scaling(부하 따라 워커 조절), **Flex**(비긴급 잡을 여유 용량으로 저비용 실행).
- **Job Bookmarks** — 이미 처리한 데이터(파일·파티션·키)를 상태로 저장 → 재실행 시 **신규 데이터만** 처리(중복 방지). 전체 재처리하려면 bookmark reset. 증분 ETL의 핵심.
- **DynamicFrame** — 스키마 온 리드(유연). `ResolveChoice` 로 타입 충돌 해결, `Relationalize` 로 중첩 구조 평탄화, `toDF()` 로 Spark DataFrame 전환. Spark 최적화가 필요하면 DataFrame으로 바꿔 처리.
- **성능** — 파티션 프루닝, **push-down predicate**(`push_down_predicate` / catalog partition predicate), 파티션 인덱스, 작은 파일 병합(`groupFiles` / `groupSize`), 컬럼 포맷(Parquet)으로 출력, 불필요 컬럼 조기 제거.

- **크롤러** — 데이터 스토어 스캔 → 카탈로그 테이블·파티션 생성/갱신. 내장·커스텀 classifier(Grok/JSON/XML/CSV). 스케줄 실행. 스키마 변경 정책(갱신 / 무시 / 새 테이블), 삭제된 객체 정책. 파티션이 규칙적이면 크롤러 대신 **파티션 프로젝트**로 비용·지연 절감.
- **Glue Workflows** — 크롤러+잡을 DAG로. 트리거(on-demand / scheduled / **event(EventBridge)** / conditional). 잡 간 파라미터·상태 공유. Glue만 쓰는 파이프라인이면 이걸로 충분.
- **Glue Data Quality** — DQDL 규칙셋(뒤 11장). 카탈로그 또는 ETL 잡 내부에서 실행, 추천 규칙 자동 생성, 결과·점수 → CloudWatch/EventBridge, 불량 행 격리.
- **Glue 연결(Connection)** — VPC 내 JDBC(RDS 등) 접근 설정, 자격 증명은 Secrets Manager. 네트워크(서브넷·SG) 포함.
- **Glue Studio / 노트북** — 시각적 잡 편집, 대화형 세션(개발 시 과금 주의).

2.3 Amazon EMR 상세

- **노드 역할** — **Primary**(클러스터 조정, 단일), **Core**(HDFS 저장 + 작업 실행), **Task**(연산만, HDFS 없음 → **Spot에 이상적**, 잃어도 데이터 손실 없음).
- **프로비저닝** — **Instance Fleets**(여러 인스턴스 타입·AZ, Spot 목표 용량, 중단 시 자동 대체) vs **Instance Groups**(그룹당 단일 타입). 중단 내성이 필요하면 Fleets.
- **스토리지** — **HDFS**(로컬 디스크, 클러스터 종료 시 소멸 = 임시), **EMRFS**(S3를 파일시스템처럼 = 영구, 여러 클러스터·서비스 공유). S3는 이제 강한 읽기 일관성이라 EMRFS consistent view는 불필요.
- **클러스터 형태** — **Transient**(잡 끝나면 자동 종료, 비용↓, 배치에 적합) vs **Long-running**(상시, 인터랙티브·다중 팀). **EMR Serverless**(용량 관리 불필요, 잡 단위) / **EMR on EKS**(쿠버네티스 위에서 Spark).
- **스케일링** — Managed Scaling(부하 기반 자동 확장/축소), Spot 중단 시 재시도. **부트스트랩 액션**으로 초기 설치·설정.
- **트랜잭션 레이크** — Apache **Hudi** / **Iceberg** / **Delta** 네이티브 지원(upsert/delete, time travel, 스키마·파티션 진화).
- **비용 절감** — Core는 On-Demand/RI로 안정 확보, Task는 Spot 다수. Transient + S3(EMRFS) 조합.

2.4 Spark 튜닝 포인트

- 셔플 파티션 수(`spark.sql.shuffle.partitions`) 조정, **브로드캐스트 조인**(작은 테이블을 각 노드에 복제해 셔플 제거), 데이터 스쿠 → **salting**(키에 랜덤 접두사).
- 작은 파일 문제 → `coalesce` / `repartition` 으로 출력 파일 수 제어, 컬럼 포맷 + 압축.
- `cache` / `persist` 로 재사용 DF 캐시, predicate/projection pushdown, 파티션 컬럼으로 필터해 스캔 최소화.
- 메모리 부족(OOM) → 워커 타입 상향(G.2X/G.8X), 파티션 잘게, 셔플 스플 확인.

⚠ 시험 포인트 — ① "서버리스 + 데이터 카탈로그 자동" = Glue. ② "이미 온프레미스 Hadoop/Spark 쓰던 팀, Spot으로 비용 최소화, HBase/Presto" = EMR. ③ "Glue 잡을 매일 돌리는데 어제 처리한 파일을 또 처리한다" = **Job Bookmark 활성화**. ④ "중첩 JSON을 관계형 테이블로" = DynamicFrame `Relationalize`. ⑤ "코드 없이 데이터 프로파일링·정제" = DataBrew.

3. 파이프라인 오케스트레이션

3.1 도구 선택

도구	특징 / 선택 기준
AWS Step Functions	상태 머신(ASL JSON). 서버리스. Standard (장기 최대 1년, 정확히 1회, 실행 이력 감사) vs Express (고빈도·최대 5분, 저비용, 최소 1회). Retry·Catch·Parallel· Map(동적 병렬) . Glue·EMR·ECS·Lambda·SNS·SQS 등 직접 통합 (<code>.sync</code> 로 완료까지 대기). 오류 처리·분기가 강점.
Amazon MWAA (Managed Airflow)	Python DAG 코드로 복잡한 의존성·백필·센서·XCom. 풍부한 오퍼레이터, 기존 Airflow 자산 이전. 스케줄러 상시 비용(환경 항상 켜짐).
AWS Glue Workflows	Glue 잡·크롤러 전용 DAG. 트리거 체인. Glue만 쓰는 파이프라인에 간단·무료.
Amazon EventBridge	이벤트 버스: 규칙(패턴 매칭) → 타겟. Scheduler (cron/rate, 대규모 일정, 유연한 시간대). Pipes (소스→필터→보강→타겟, point-to-point). 스키마 레지스트리. 서비스 간 느슨한 결합.
Amazon SNS / SQS	SNS: 팬아웃(이메일/HTTP/SQS/Lambda), 필터 정책. SQS: 버퍼링·재시도·DLQ·백프레셔. 파이프라인 알림·디커플링.

판단 규칙: 복잡한 의존성·백필·센서·팀이 Airflow에 익숙 → MWAA. 서버리스·상태별 재시도/분기·AWS 서비스 오케스트레이션 → Step Functions. Glue만 → Glue Workflows. 이벤트 라우팅·스케줄만 → EventBridge.

3.2 Step Functions 세부

- **상태 타입** — Task, Choice(분기), Parallel(정적 병렬), **Map**(배열마다 반복, Distributed Map은 수십만 항목·S3 대상), Wait, Pass, Succeed/Fail.
- **오류 처리** — `Retry` (백오프·최대 시도), `Catch` (실패 시 대체 경로), 타임아웃. 부분 실패를 격리하고 알림(SNS)까지 상태 머신 안에서.
- **서비스 통합** — `arn:aws:states:::glue:startJobRun.sync` 처럼 `.sync` 를 붙이면 해당 잡이 끝날 때까지 대기 후 다음 상태로.
- **Standard vs Express** — 이력·감사·장기 실행 = Standard. 초당 수천 건 이벤트 처리·짧은 워크플로 = Express(비용 크게↓).

3.3 패턴

- **이벤트 기반** — S3 도착 → EventBridge 규칙 → Step Functions → (Glue 크롤러 → Glue 잡 → DQ 체크) → 완료 이벤트 → 다운스트림.
- **스케줄 기반** — EventBridge Scheduler(권장, 수백만 스케줄·타임존) / Airflow cron / Glue 스케줄 트리거.
- **소스→타겟 단순 연결** — EventBridge **Pipes**(예: DynamoDB Streams → 필터 → Lambda 보강 → Step Functions).
- **재시도·내결합성** — 멍등성 설계(같은 입력 두 번 처리해도 결과 동일), 체크포인트, DLQ, 부분 실패 격리, 실패 알림.

⚠ 시험 포인트 — ① "복잡한 DAG·백필·다양한 오퍼레이터·기존 Airflow" → MWAA. ② "서버리스, 각 단계 재시도·에러 분기, Glue/EMR 완료 대기" → Step Functions(+ `.sync`). ③ "수백만 개 예약 작업, 타임존별" → EventBridge Scheduler. ④ Express Workflow는 최대 5분·최소 1회(역등성 필수).

4. 파일 포맷 · 압축 · 파티셔닝 (성능·비용의 핵심)

4.1 포맷 비교

포맷	유형	특징
Parquet	컬럼형	분석 표준. 높은 압축, 컬럼 프루닝, predicate pushdown, splittable. Athena/Spectrum/Spark 최적.
ORC	컬럼형	Hive 최적, 내장 인덱스·통계(스트라이프), 높은 압축.
Avro	행형	스키마 진화 우수, 스키마를 파일에 동봉, 스트리밍·쓰기 위주·직렬화.
JSON / CSV	행형(텍스트)	가독성↑, 비효율(전체 스캔). CSV는 스키마·타입 없음. 원천 수집 단계에 혼함.

- 읽기(분석) 중심 → Parquet/ORC, 쓰기·스키마 진화·스트림 직렬화 → Avro, 원본 수집 → JSON/CSV 그대로 두고 이후 변환.

4.2 압축 — splittable 여부가 핵심

압축	특징	splittable
Snappy	빠름, 압축률 중간. Parquet 내부 기본	예(Parquet 블록 단위)
gzip	압축률↑, CPU↑	아니오(텍스트 파일이면 1개 매퍼가 통째로)
bzip2	압축률 매우↑, 매우 느림	예
Zstd	압축률·속도 균형(권장 추세)	예
LZO	빠름, 인덱스하면 splittable	조건부

⚠ 함정 — 큰 CSV/JSON을 **gzip**으로 두면 Spark/EMR에서 파일 하나를 한 태스크가 전부 읽어 병렬성이 죽는다. 대용량 텍스트는 bzip2/Zstd 또는 여러 파일로 쪼개거나, 아예 Parquet+Snappy로 변환.

4.3 비용 절감 공식 (Athena/Spectrum)

컬럼형(Parquet) + 파티셔닝 + 압축 + 적정 파일 크기(≈128 MB~1 GB). Athena·Spectrum은 **스캔한 바이트**로 과금하므로, 이 네 가지가 곧 비용 절감이다. `SELECT *` 지양, 필요한 컬럼만.

4.4 파티셔닝

- S3 프리픽스를 Hive 스타일로: `s3://bucket/table/year=2024/month=01/day=05/`. 쿼리 `WHERE year=2024 AND month=1` 이면 **파티션 프루닝**으로 해당 경로만 스캔.
- 파티션 키는 **자주 필터하는 저카디널리티 컬럼**(날짜, 리전). 과도하게 잘게 나누면 작은 파일 폭증.
- **Athena Partition Projection** — 파티션 메타데이터를 카탈로그에 넣지 않고 테이블 속성의 규칙(범위·형식)으로 계산
→ 파티션 수백만 개, 크롤러 부담. `ADD PARTITION` 지연 해소.

4.5 작은 파일 문제

증상: 파일 수백만 개 → 메타데이터·오프닝 오버헤드로 쿼리 느림·비용↑. 해결: - **Compaction** 잡(Glue/EMR로 주기적으로 합치기), Firehose **버퍼 크기 상향**, Glue `groupFiles / groupSize`, Spark `coalesce`. - **Iceberg/Hudi**의 자동 compaction·클러스터링.

4.6 트랜잭션 테이블 포맷 (레이크하우스)

	Iceberg	Hudi	Delta
강점	스키마·파티션 진화, 숨은 파티셔닝, 엔진 중립 (Athena·EMR·Spark·Flink·Trino)	빠른 upsert, 인덱스, CDC 친화 (MoR/CoW)	Spark 생태계, Databricks 기원
AWS 통합	Athena·Glue·EMR·Redshift 폭넓게	EMR·Glue	EMR·Glue

공동 이점: ACID, **row-level update/delete/merge**, time travel, 스키마 진화. "S3 데이터 레이크에서 GDPR 삭제·행 수정·업서트가 필요" → 이들 포맷(시험에서 자주 **Iceberg**).

⚠ 시험 포인트 — ① Athena 비용↓ = Parquet + 파티션 + 압축 + `SELECT` 컬럼 제한. ② 파티션 폭증·크롤러 느림 = Partition Projection. ③ 레이크에서 개별 레코드 삭제/수정 = Iceberg(또는 Hudi/Delta). ④ gzip 텍스트 = splittable 아님.

5. 데이터 스토어 선택 · S3 수명주기

5.1 요구사항 → 스토어

요구	적합 서비스
데이터 레이크 원본·스테이징	Amazon S3 (+ Glue Catalog / Lake Formation)
대규모 정형 분석·DW·BI	Amazon Redshift (RA3 / Serverless)
애드혹 S3 SQL 쿼리(서버리스)	Amazon Athena
초저지연 키-값 / 고 TPS 앱	Amazon DynamoDB (+ DAX)

요구	적합 서비스
OLTP 관계형	Amazon RDS / Aurora
실시간 스트림 버퍼	Kinesis Data Streams / MSK
검색·로그 분석·관측	Amazon OpenSearch Service
인메모리 캐시	ElastiCache / MemoryDB / DAX
그래프 / 시계열 / 원장	Neptune / Timestream / QLDB

5.2 Amazon S3

- **스토리지 클래스** — Standard → Standard-IA / One Zone-IA(재생성 가능·비중요) → **Intelligent-Tiering**(접근 패턴 모를 때 기본, 자동 계층화, 모니터링 수수료) → Glacier Instant(즉시) / Flexible(분~시간) / Deep Archive(최저가, 12h+ 복원).
- **Lifecycle 정책** — N일 후 클래스 전환, M일 후 만료(삭제), 미완료 멀티파트 업로드 정리, 비현행 버전 관리. 로그·백업 비용 관리의 기본.
- **버전 관리** + MFA Delete, **Object Lock**(WORM: Compliance 모드는 루트도 해제 불가 / Governance 모드는 권한 있으면 해제) = 보존·불변·규정 준수.
- **S3 Select** — 객체 내부에서 SQL로 필요한 행·열만 반환(단일 객체, CSV/JSON/Parquet). 대량·조인은 Athena.
- **Storage Lens** — 계정·조직 전체 사용량·비용·이상 가시성. **S3 Inventory** — 객체 목록·메타데이터 리포트(암호화 상태 감사 등).
- **복제** — CRR(교차 리전, DR·지연·레지던시) / SRR(동일 리전, 로그 집계·계정 분리). RTC로 복제 SLA. 규칙으로 특정 리전 복제 차단.
- **성능·일관성** — 프리픽스별로 자동 확장(요청 분산), **강한 읽기-쓰기 일관성**(2020~). 전송 가속(엣지), 요청자 지불.
- **접근 제어** — 버킷 정책 + **ACL 비활성**(Object Ownership = Bucket owner enforced, 권장), S3 Access Points(용도별 정책), 퍼블릭 액세스 차단(계정·버킷).



DynamoDB TTL / S3 만료로 수명주기·비용 관리. Redshift ↔ S3 이동은 COPY / UNLOAD .



시험 포인트 — ① "접근 패턴을 모른다" = Intelligent-Tiering. ② "규정상 일정 기간 삭제·수정 불가" = Object Lock(Compliance). ③ "가장 저렴, 복원 몇 시간 OK" = Glacier Deep Archive. ④ "한 리전에만 있어야 한다(데이터 레지던시)" = 복제 규칙 제한 + SCP.

6. Amazon Redshift 심화

6.1 아키텍처·노드

- **노드 타입** — RA3(관리형 스토리지 = 컴퓨터·스토리지 분리, 권장) / DC2(로컬 SSD, 소규모·저지연) / **Serverless**(RPU 단위 자동, 간헐·예측 불가 워크로드).

- MPP(리더 노드 + 컴퓨트 노드 → 슬라이스), 컬럼형 저장, **존 맵**(블록별 min/max로 불필요 블록 스킵).

6.2 분산 스타일(DISTSTYLE)

스타일	동작	언제
KEY	지정 컬럼 해시로 같은 값은 같은 노드	큰 팩트-팩트/팩트-디멘전 조인 키
ALL	전 노드에 테이블 복제	작은 디멘전(수십만 행 이하)
EVEN	라운드로빈	조인 안 하는 스테이징·중간 테이블
AUTO	Redshift가 크기 보고 자동 선택	기본값, 잘 모르면

6.3 정렬 키(SORTKEY)

- **Compound**(기본) — 프리픽스 컬럼 순서가 중요. 범위/동등 필터, `GROUP BY`, `ORDER BY` 에 효과. 보통 **날짜 컬럼**을 첫 SORTKEY로.
- **Interleaved** — 여러 컬럼에 동등 가중, 유지비(VACUUM REINDEX)↑ → 최근엔 비권장.
- 존 맵 + SORTKEY로 스캔 블록을 크게 줄인다.

6.4 적재/추출

- **COPY** — S3/EMR/DynamoDB/SSH에서 **병렬** 적재. 파일 수는 슬라이스 수의 배수로 분할(가장 자주 나오는 튜닝), gzip/bzip2/zstd 압축, **매니페스트**로 정확한 파일 목록·정합성, `COMPUPDATE` / `STATUPDATE`.
- **UNLOAD** — 쿼리 결과를 S3로 병렬 내보내기(Parquet·파티션 지정 가능). BI 추출·레이크 공유.
- 단건 `INSERT` 반복은 금물(느림) — 스테이징 테이블 + `COPY` + `MERGE`.

6.5 워크로드 관리

- **WLM** — 쿼리 큐·메모리·동시성. **Auto WLM**(권장) vs Manual. **SQA**(짧은 쿼리 우선 큐), **Concurrency Scaling**(읽기 폭주 시 임시 클러스터 자동 추가, 크레딧), **QMR**(규칙으로 폭주 쿼리 중단·강등).

6.6 레이크·외부 연동

- **Redshift Spectrum** — S3 외부 테이블 직접 쿼리(Glue Catalog 스키마). **스캔량 과금**, 컴퓨트는 Redshift와 별개 풀. 콜드 데이터는 S3, 핫 데이터는 Redshift에 두고 조인.
- **Federated Query** — RDS/Aurora(PostgreSQL·MySQL) **라이브** 쿼리(이동 없이).
- **Materialized View** — 사전 계산 결과, 자동 새로고침·자동 쿼리 재작성. 반복 집계 가속.
- **Data Sharing(RA3)** — 클러스터·계정·리전 간 **복제 없이 라이브 읽기 공유**(프로듀서/컨슈머). AWS Data Exchange for Redshift로 외부 판매·구독.

6.7 유지관리·최신 기능

- **VACUUM**(삭제 공간 회수·재정렬), **ANALYZE**(통계 갱신) — 대개 자동. 대규모 재구성은 deep copy.

- **Zero-ETL**(Aurora → Redshift 자동 복제), **Streaming Ingestion**(Kinesis/MSK → 물리 뷰), **Auto-copy**(S3 이벤트 → 자동 COPY), **Redshift ML**(SQL로 SageMaker 모델), **동적 데이터 마스킹**, **행 수준 보안(RLS)**.

6.8 보안·관측

- KMS 저장 암호화, VPC 내 배치·**Enhanced VPC Routing**(COPY/UNLOAD 트래픽을 VPC로), IAM 인증·IAM Identity Center, 감사 로그 → S3/CloudWatch, 시스템 뷰 **STL/SVL/SVV**(쿼리·큐·디스크 분석).

⚠ 시험 포인트 — ① 큰 팩트-팩트 조인 = 조인 컬럼에 **DISTKEY**. ② 작은 디멘전 = **DISTSTYLE ALL**. ③ 날짜 범위 필터 = 날짜 **SORTKEY**. ④ COPY 느림/불균형 = 파일을 슬라이스 배수로 분할. ⑤ 낮 시간 대시보드 동시성 폭주 = **Concurrency Scaling** + **Auto WLM**. ⑥ S3 콜드 데이터 조인 = Spectrum. ⑦ 다른 팀에 복제 없이 데이터 제공 = Data Sharing.

7. Amazon DynamoDB 심화

7.1 키 설계

- **기본 키** — 파티션 키(해시) 단독 또는 파티션 키 + 정렬 키(복합). 파티션 키 **카디널리티가 높고 접근이 고르게 분산**되어야 한다.
- 파티션당 물리 한계 ≈ **3,000 RCU / 1,000 WCU**. 특정 키에 트래픽 몰림 = **핫 파티션** → **스로틀링**.

7.2 용량 모드

	On-demand	Provisioned
언제	트래픽 예측 불가·스파이크·신규·개발	꾸준·예측 가능(+ Auto Scaling)
비용	요청당(비쌈), 관리 0	예약 용량(쌈), 관리 필요

- **RCU** = 강한 일관성 4 KB 읽기/초(최종 일관성은 2배, 트랜잭션은 절반). **WCU** = 1 KB 쓰기/초(트랜잭션은 절반).

7.3 핫 파티션·스로틀링 대응

- 고엔트로피 파티션 키, **write sharding**(키에 #1 ~ #N 접미사 후 분산 write, 읽을 때 병합), 시계열은 **테이블/파티션을 기간별로 분리**, **adaptive capacity**(자동으로 핫 파티션에 용량 재배분 — 자동이지만 만능 아님), 버스트 용량.

7.4 인덱스

	GSI	LSI
키	다른 파티션 + 정렬 키	같은 파티션 + 다른 정렬 키
생성	언제든 추가/삭제	테이블 생성 시에만

	GSI	LSI
용량	별도 프로비저닝	테이블 용량 공유
일관성	최종 일관성만	강한 일관성 가능
제한	기본 20개/테이블	5개, 아이템 컬렉션 10 GB

- 프로젝션 속성(KEYS_ONLY / INCLUDE / ALL)으로 인덱스 크기·비용 조절.

7.5 부가 기능

- DAX** — 완전관리형 인메모리 캐시(마이크로초 읽기, read-through/write-through). 읽기 폭주·반복 조회.
- TTL** — 만료 타임스탬프 속성으로 자동 삭제(무료, 최대 ~48h 내). 삭제 이벤트도 Streams에 기록 → 아카이브 파이프라인.
- Streams** → Lambda 트리거 / **Kinesis Data Streams for DynamoDB**(더 긴 보존·팬아웃·다중 소비자).
- Global Tables** — 다중 리전 액티브-액티브(마지막 쓰기 우선). **PITR**(연속 백업 35일), 온디맨드 백업.
- 분석 연동** — **S3로 Export**(PITR 기반, 읽기 용량 소비 없음, 테이블 영향 0) → Athena/Glue/Redshift Spectrum로 분석. Import from S3.

7.6 데이터 모델링

- 액세스 패턴을 먼저 정의**하고 그에 맞게 설계(관계형처럼 정규화 후 조인 X). **단일 테이블 디자인**, 오버로디드 GSI(GSI 키를 범용 이름으로), 비정규화·중복 허용. 트랜잭션은 `TransactWriteItems` / `TransactGetItems` (항목 최대 100, 4 MB).

⚠ 시험 포인트 — ① 특정 키에 스토리징 = 파티션 키 카디널리티↑ + write sharding. ② 스파이크성·예측 불가 = **On-demand**. ③ 인덱스를 나중에 추가해야 함 = **GSI**(LSI는 생성 시에만). ④ DynamoDB 데이터를 분석 = **S3 Export**(용량 영향 없음) 후 Athena. ⑤ 마이크로초 읽기 캐시 = **DAX**.

8. Athena · Glue Data Catalog · Lake Formation

8.1 Amazon Athena

- 서버리스 **Trino/Presto(SQL)** + Athena for Apache Spark. S3 데이터를 Glue Catalog 스키마로 쿼리, 관리 서버 없음. **엔진 v3**.
- 과금** — 스캔한 데이터 **TB당 약 \$5**(리전별 상이). 압축·컬럼형·파티셔닝으로 절감. Spark는 DPU 기반.
- Workgroup** — 쿼리별/그룹별 **데이터 스캔 한도**(초과 시 취소), 비용 태그, 결과 위치·암호화 격리, 별도 CloudWatch 지표. 팀·환경 분리와 비용 통제의 핵심.
- CTAS** / **INSERT INTO** — 쿼리 결과로 새 테이블 생성(포맷·파티션·압축 지정) → 경량 ETL·포맷 변환. **UNLOAD** 로 포맷 지정 내보내기.
- Partition Projection, Query result reuse**(동일 쿼리 결과 캐시), **Prepared statements**, UDF(Lambda).

- **Federated Query** — Lambda 데이터 소스 커넥터로 DynamoDB·RDS·Redshift·CloudWatch Logs·온프레미스 등을 조인.
- **ACID·행 수준 업데이트 필요** → **Apache Iceberg** 테이블(Athena가 네이티브 지원, **MERGE** ·time travel).

8.2 AWS Glue Data Catalog

- 중앙 메타데이터 저장소(**Hive 메타스토어 호환**). Athena·EMR·Redshift Spectrum·Glue·Lake Formation·QuickSight가 공유.
- 구조: **Database** → **Table**(스키마, S3 위치, 포맷, SerDe, 파티션). **파티션 인덱스**로 대규모 파티션에서 조회 성능.
- 파티션 동기화 방법: 크롤러 / **MSCK REPAIR TABLE** / **ALTER TABLE ADD PARTITION** / **파티션 프로젝션**.
- **Schema Registry**(Avro/JSON/Protobuf) — 스트리밍 프로듀서/컨슈머 간 스키마 호환성 강제 (BACKWARD/FORWARD/FULL).

8.3 AWS Lake Formation

- 레이크 **중앙 권한 관리** — DB/테이블/컬럼·행·셀 수준 권한. IAM 위에 있는 세분화 레이어(먼저 Lake Formation을 통해 위임).
- **LF-Tags**(태그 기반 접근 제어, **TBAC**) — 리소스와 주체에 태그를 붙여 대규모 권한을 선언적으로 관리. **Data filters**(열 선택 + 행 필터식).
- **교차 계정 공유**(태그 또는 명명 리소스), 통합 대상: Athena, Redshift Spectrum, EMR, Glue, QuickSight.
- **Blueprints**(수집 워크플로 템플릿, 데이터 위치 등록·권한, **데이터 리니지**, **Hybrid access mode**(IAM 직접 접근과 LF 관리 병행)).

⚠ 시험 포인트 — ① "여러 팀이 같은 카탈로그, 팀마다 특정 컬럼·행만" = Lake Formation(LF-Tags / Data filters). ② "Athena 비용을 팀별 상한으로" = Workgroup 스캔 한도. ③ "쿼리 결과를 파티션된 Parquet 테이블로" = CTAS. ④ "S3 레이크에서 **UPDATE** / **DELETE**" = Iceberg. ⑤ 스트리밍 스키마 깨짐 방지 = Glue Schema Registry 호환성 모드.

9. 데이터 모델링 · 스키마 진화 · 리니지

- **모델링** — 정규화(OLTP, 중복 제거·쓰기 최적) vs **차원 모델링/스타 스키마**(팩트 + 디멘전, 읽기·집계 최적, DW). 스노우플레이크 스키마는 디멘전을 더 정규화.
- **SCD(느리게 변하는 디멘전)** — Type 1(덮어쓰기, 이력 없음) · **Type 2**(변경마다 새 행 + 유효기간/현재 플래그, 이력 보존) · Type 3(이전 값 컬럼 하나).
- **스토어별** — Redshift: 팩트 = DISTKEY(조인 키) + SORTKEY(날짜), 디멘전 = DISTSTYLE ALL, 서로게이트 키. DynamoDB: 액세스 패턴 기반 단일 테이블·비정규화. S3 레이크: **존 분리**(raw → cleansed/conformed → curated), Parquet + 파티션.
- **스키마 진화** — Avro(가장 유연) > Parquet(컬럼 추가 대체로 OK) > CSV(취약). **Iceberg/Hudi/Delta**는 스키마·파티션 진화 + time travel. Glue 크롤러 스키마 변경 정책, Schema Registry 호환성 모드.
- **스키마 변환(이기종 이전)** — AWS SCT, DMS Schema Conversion.

- **데이터 리니지** — Glue 데이터 리니지, SageMaker/DataZone 계보, DataBrew 계보 뷰. "데이터가 어디서 와서 어떻게 가공됐는지" 추적·감사.
- **물리 최적화** — 파티셔닝·버킷팅, Redshift 컬럼 인코딩(AZ64/ZSTD), 파티션 인덱스·RDS 인덱스, 통계 최신화.

⚠ **시험 포인트** — ① 디멘전 변경 이력을 모두 보관 = **SCD Type 2**. ② 분석 DW 설계 = 스타 스키마(팩트+디멘전). ③ 나중에 컬럼이 자주 추가됨 = Avro / Iceberg. ④ raw는 그대로 두고 정제본을 따로 = 레이크 존 분리.

10. 운영 · 모니터링 · 로깅

- **Amazon CloudWatch** — 지표·알람·대시보드. **Logs**(수집·보존 정책), **Logs Insights**(쿼리 언어로 로그 분석), **Metric Filter**(로그 패턴 → 커스텀 지표 → 알람), Contributor Insights, EMR/Glue/Kinesis/Redshift 내장 지표.
- **AWS CloudTrail** — 모든 API 호출 감사. **관리 이벤트**(기본, 무료 1부) + **데이터 이벤트**(S3 객체 수준 GET/PUT, Lambda invoke — 별도 활성화·과금). **CloudTrail Lake**(SQL로 감사 쿼리, 최대 10년 보존). **조직 트레일**(전 계정 → 중앙 S3).
- **핵심 지표(외우기)** — Kinesis `GetRecords.IteratorAge` (소비 지연), `Write/ReadProvisionedThroughputExceeded`. DynamoDB `ThrottledRequests`. `ConsumedRead/WriteCapacityUnits`. `SystemErrors`. Redshift `CPUUtilization`. `PercentageDiskSpaceUsed`. `QueueLength`. `WLMQueueWaitTime`. Glue 잡 `glue.driver.aggregate.*`, DPU 사용·실행시간.
- **Glue 잡 관측** — Continuous logging(CloudWatch로 실시간 로그), Spark UI / History Server, **Job Run Insights**(실패 원인·권장), 잡 지표.
- **EMR** — CloudWatch 지표, YARN·Spark History·Persistent App UI, S3 로그 아카이브(부트스트랩·스텝 로그).
- **로그 분석 패턴** — ① 로그를 S3로 → Athena/Glue로 SQL. ② OpenSearch로 인덱싱(대시보드·검색). ③ CloudWatch Logs Insights(빠른 조사). 대용량 배치 분석은 EMR.
- **알림·자동 조치** — CloudWatch 알람 → SNS. 이벤트 → EventBridge → Lambda/Step Functions로 자동 복구·재처리.

⚠ **시험 포인트** — ① 스트림 소비자가 뒤처지는지 = `IteratorAge` 알람. ② S3 객체 단위 접근 감사 = CloudTrail **데이터 이벤트** 활성화. ③ 로그에서 특정 에러 발생 횟수를 지표·알람으로 = **Metric Filter**. ④ 여러 계정 감사 로그 한곳에 = 조직 트레일 → 중앙 S3(+ CloudTrail Lake).

11. 데이터 품질

- **검증 차원** — 완전성(결측 없음), 유일성(중복 없음), 유효성(형식·범위·허용값), 정확성, 일관성(교차 필드·소스 간), 적시성(freshness), 참조 무결성.
- **AWS Glue Data Quality** — **DQDL** 규칙셋 작성. 예:
 - `Completeness "email" > 0.95`
 - `IsUnique "order_id"`

- ColumnValues "status" in ["NEW", "PAID", "SHIPPED"]
- RowCount between 1000 and 5000
- Freshness "updated_at" <= 24 hours

카탈로그 테이블 또는 ETL 파이프라인 내에서 실행, **추천 규칙 자동 생성**, 점수·결과 → EventBridge/CloudWatch, 불량 레코드 분리(격리 경로). - **Glue DataBrew** — 데이터 프로파일링(통계·분포·상관·이상치·유효성), 250+ 정제 변환, 데이터 품질 규칙, 재사용 레시피. 코드 없이. - **Deequ**(EMR/Spark 라이브러리) — 대규모 데이터의 "유닛 테스트"식 제약 검증·이상 탐지(대용량·프로그래매틱). - **파이프라인 게이트** — 규칙 위반 시 잡 실패 / 격리 / 알림. 배포 전 품질 체크를 필수 단계로.

⚠ 시험 포인트 — ① "ETL 파이프라인에 규칙 기반 품질 검사, 위반 시 알림·격리" = **Glue Data Quality(DQDL)**. ② "분석가가 코드 없이 프로파일링·정제" = **DataBrew**. ③ "EMR/Spark에서 대규모 제약 검증" = **Deequ**.

12. 데이터 보안 (인증 · 인가 · 암호화)

12.1 IAM

- 자격 증명 기반 vs **리소스 기반 정책**(S3 버킷 정책, KMS 키 정책, Lambda 리소스 정책 — 교차 계정에 필수), **역할**(교차 계정·서비스 간, 임시 자격 증명), 최소 권한, 조건 키(`aws:SourceIp` , `aws:SecureTransport` , `s3:prefix` 등), **권한 경계**, **ABAC**(태그 기반). AWS 관리형 vs 고객 관리형 정책.

12.2 인증 방식

- 암호 기반 · 인증서 기반 · **역할(임시 자격 증명, 권장)**. Redshift/RDS는 **IAM 인증**·IAM Identity Center 연동 가능. 애플리케이션은 인스턴스 프로파일/실행 역할 사용(키 하드코딩 금지).

12.3 시크릿 관리

	Secrets Manager	SSM Parameter Store
자동 회전	있음(Lambda, RDS/Redshift/DocumentDB 네이티브)	없음
비용	시크릿당 유료	표준 무료(고급 파라미터 유료)
용도	DB 자격 증명·API 키(회전 필요)	설정값·플래그, SecureString(KMS)

12.4 저장 데이터 암호화 (S3)

방식	설명
SSE-S3	S3 관리 AES-256 키. 가장 간단(현재 기본 활성화).
SSE-KMS	KMS 키 사용 → 키 정책·권한 분리· CloudTrail 감사 ·회전. 고객 관리형 CMK로 세밀 제어. S3 Bucket Key 로 KMS 요청 수·비용↓.

방식	설명
DSSE-KMS	이중 계층 KMS 암호화(강한 규제 요건).
SSE-C	고객이 키 제공, AWS는 키 저장 안 함(키 관리 책임 고객).
CSE	클라이언트 측 암호화(업로드 전). AWS는 평문 미접근.

12.5 전송 중 암호화

- HTTPS/TLS 강제 — S3 버킷 정책에 `aws:SecureTransport = false` 거부. Redshift/RDS SSL, JDBC/ODBC SSL, VPC 내부 트래픽.

12.6 KMS

- **봉투 암호화**(데이터 키로 데이터 암호화, 데이터 키는 CMK로 암호화). 키 정책 + IAM + **grant**. 자동 연간 회전(대칭 CMK), **멀티 리전 키**, 교차 계정 키 부여. 분석 서비스(Redshift/EMR/Glue/Kinesis/Athena 결과) 전부 KMS 통합. CloudHSM은 전용 HW가 필요할 때.

12.7 네트워크 격리

- **VPC Gateway Endpoint**(S3·DynamoDB 전용, **무료**, 라우트 테이블 기반) / **Interface Endpoint(PrivateLink)**(그 외 대부분 서비스, ENI, 유료). 프라이빗 서브넷 + NAT, 보안 그룹·NACL, **S3 Access Points**(경로별 정책). Glue/EMR/Redshift/MSK를 VPC에 배치.

12.8 인가(권한)

- 최소 권한, RBAC/ABAC, **Lake Formation**(컬럼·행·셀), Redshift **GRANT** (사용자·그룹·역할·RLS), S3 버킷 정책 + ACL 비활성.

⚠ 시험 포인트 — ① 감사·회전·키별 권한 필요 = **SSE-KMS**(+ 요청 많으면 Bucket Key). ② DB 자격 증명 자동 회전 = **Secrets Manager**. ③ S3·DynamoDB 프라이빗 접근(무료) = **Gateway Endpoint**, 그 외 = Interface Endpoint. ④ "HTTPS 아닌 요청 차단" = 버킷 정책 `aws:SecureTransport`. ⑤ 교차 계정 S3 접근 = 리소스 기반 정책(버킷 정책) + 역할.

13. 데이터 거버넌스 · 프라이버시 · 감사

- **PII 탐지** — **Amazon Macie**(S3 민감 데이터 자동 발견·ML 분류, findings → EventBridge/Security Hub). **Glue 민감 데이터 탐지**(PII detection transform — ETL 중 탐지·마스킹·삭제·해시). Amazon Comprehend(NLP 엔터티). Macie + Lake Formation 연계.
- **마스킹·익명화** — Redshift **동적 데이터 마스킹**·RLS, Lake Formation 컬럼 권한·데이터 필터, Athena/Glue 뷰·변환, 토큰화·해싱·키 솔팅, k-익명성/일반화.

- **데이터 주권·레지던시** — 리전 제한 **SCP**, S3 복제 규칙으로 비허용 리전 복제 차단, KMS 리전 키, **AWS Config** 규칙으로 위반 자동 감지.
- **AWS Config** — 리소스 구성 이력·규칙(비준수 감지)·**자동 교정(remediation, SSM 문서)**. 계정 전반의 변경 추적.
- **데이터 공유** — Redshift **datashare**, Lake Formation 교차 계정, **AWS Data Exchange**(데이터 상품 구독·판매), **S3 Access Grants**(디렉터리 그룹 ↔ S3 프리픽스 권한), **AWS Clean Rooms**(여러 당사자 공동 분석, **원본 미공유**).
- **보존·불변** — S3 **Object Lock**(WORM), S3 Glacier **Vault Lock**, **AWS Backup**(중앙 백업·정책·교차 리전/계정), 수명주기 만료로 합법적 삭제(법적 보존 기간 후).
- **감사 로그 중앙화** — CloudTrail 조직 트레일 → 중앙 S3(+ **CloudTrail Lake**), CloudWatch Logs 교차 계정, 로그를 Athena/OpenSearch로 분석.

⚠ 시험 포인트 — ① "S3에 신용카드·주민번호가 있는지 자동으로 스캔" = **Macie**. ② "ETL 중 PII를 마스킹/삭제" = **Glue 민감 데이터 탐지**. ③ "여러 회사가 데이터를 합쳐 분석하되 원본은 공유 안 함" = **Clean Rooms**. ④ "리전 밖으로 데이터가 못 나가게" = SCP + S3 복제 규칙 + Config 규칙. ⑤ "리소스가 규정 위반 설정이면 자동 수정" = AWS Config remediation.

14. 자주 나오는 시나리오 · 암기 포인트

수집

- 코드 없이 스트림 → S3(포맷 변환) : **Kinesis Data Firehose** (+ Lambda 변환, Parquet 변환, 동적 파티셔닝).
- 1초 미만 실시간 + 다중 소비자 + 재처리 : **Kinesis Data Streams** (+ Enhanced Fan-Out).
- 소비자가 뒤처짐 : **GetRecords.IteratorAge** 확인 → 샤드/소비자 확장, EFO.
- 처리량 초과 예외 : 샤드 추가(split) / 파티션 키 분산 / On-demand 모드.
- DB 지속 복제(CDC)를 레이크로 : **DMS** → **S3/Kinesis**. 이기종 스키마 : **SCT**. 무파이프라인 : **Zero-ETL(Aurora→Redshift)**.
- SaaS 데이터 수집 : **AppFlow**. 온프레미스 대량 파일 : **DataSync**. 외부 파트너 업로드 창구 : **Transfer Family(SFTP)**.

변환·처리

- 서버리스 Spark ETL + 카탈로그 : **AWS Glue**. 초대형·튜닝·Spot·HBase/Presto/Hudi/Iceberg : **EMR**.
- Glue 잡 재실행 시 중복 방지 : **Job Bookmark**.
- 중첩 JSON → 관계형 : **DynamicFrame Relationalize**.
- 한 파일·수 초 처리 : **Lambda**. 무한 스트림 윈도우 집계 : **Managed Service for Apache Flink**.
- 코드 없는 정제·프로파일링 : **DataBrew**.

오케스트레이션

- 복잡한 의존성·백필·기존 Airflow : **MWAA**. 서버리스·단계별 재시도/분기·서비스 완료 대기 : **Step Functions(.sync)**. Glue 전용 : **Glue Workflows**.
- 대규모 예약(타임존별) : **EventBridge Scheduler**. 소스→필터→보강→타겟 : **EventBridge Pipes**.

포맷·레이크

- Athena/Spectrum 비용↓ : **Parquet + 파티션 + 압축 + 컬럼 선택**. 과금은 스캔 바이트.
- 파티션 수백만·크롤러 부담 : **Partition Projection**.
- 레이크에서 행 **UPDATE / DELETE / MERGE**, time travel : **Iceberg**(또는 Hudi/Delta).
- 작은 파일 과다 : compaction / Firehose 버퍼↑ / Glue **groupFiles** / Iceberg 컴팩션.
- gzip 텍스트는 **splittable 아님** → bzip2/Zstd 또는 Parquet.

스토어

- 접근 패턴 모름 : **S3 Intelligent-Tiering**. 최저가·복원 지연 OK : **Glacier Deep Archive**.
- 규정상 삭제·수정 불가 : **S3 Object Lock (Compliance)**.
- 초저지연 키·값·고 TPS : **DynamoDB (+ DAX)**. 애드혹 S3 SQL : **Athena**. 대규모 정형 DW : **Redshift**.

Redshift

- 큰 팩트·팩트 조인 : 조인 키에 **DISTKEY**. 작은 디멘전 : **DISTSTYLE ALL**. 날짜 필터 : **SORTKEY**.
- COPY 느림 : 파일을 **슬라이스 수의 배수**로 분할, 압축, 매니페스트.
- 낮 시간 읽기 동시성 폭주 : **Concurrency Scaling + Auto WLM**.
- S3 콜드 데이터 조인 : **Spectrum**. RDS 라이브 조인 : **Federated Query**. 복제 없이 팀 간 공유 : **Data Sharing**.
- Aurora 데이터를 준실시간 분석 : **Zero-ETL**.

DynamoDB

- 스로틀/핫키 : 고카디널리티 파티션 키 + **write sharding**, On-demand.
- 인덱스를 나중에 추가 : **GSI**(LSI는 테이블 생성 시만).
- 분석 : **S3로 Export**(용량 소비 없음) → Athena/Glue.
- 마이크로초 읽기 캐시 : **DAX**. 만료 자동 삭제 : **TTL**.

보안·거버넌스

- S3 민감정보 자동 발견 : **Macie**. ETL 중 PII 마스킹 : **Glue 민감 데이터 탐지**.
- KMS로 감사·회전·권한 분리 : **SSE-KMS**(+ Bucket Key). 자격 증명 자동 회전 : **Secrets Manager**.
- S3·DynamoDB 프라이빗 접근(무료) : **Gateway VPC Endpoint**. 그 외 : **Interface Endpoint(PrivateLink)**.
- 리전 밖 복제 방지 : **SCP + S3 복제 규칙 + AWS Config 규칙**.
- 컬럼·행 수준 접근 제어 : **Lake Formation (LF-Tags / Data Filters)**.
- 공동 분석, 원본 비공유 : **Clean Rooms**.
- 규정 위반 리소스 자동 교정 : **AWS Config remediation**.

운영·품질

- 스트림 소비 지연 : **IteratorAge**. S3 객체 단위 감사 : **CloudTrail 데이터 이벤트**.
- 로그에서 에러 횟수를 알람으로 : **CloudWatch Metric Filter**.
- 파이프라인 품질 규칙 : **Glue Data Quality (DQDL)**.

- 여러 계정 감사 로그 집계 : 조직 트레일 → 중앙 S3 (+ CloudTrail Lake).

15. 마지막 점검 — 헛갈리는 짝 비교

상황	A	B	가르는 기준
스트림 적재	Firehose	KDS	코드·재생 불필요·S3/DW 적재 → Firehose · 1초 미만·다중 소비자·replay → KDS
큐	SQS	SNS	작업 분산·버퍼 → SQS · 팬아웃 알림 → SNS · 스트림 분석 → Kinesis
ETL 엔진	Glue	EMR	서버리스·카탈로그·중규모 → Glue · 초대형·튜닝·Spot·특수엔진 → EMR
오케스트레이션	Step Functions	MWAA	서버리스·상태별 에러 처리 → SFn · 복잡 DAG·백필·Airflow 자산 → MWAA
스키마 변환	SCT	DMS	스키마·코드 변환(이기종) → SCT · 데이터 이동·CDC → DMS
대량 파일 이동	DataSync	Transfer Family	대량 복사·동기화 엔진 → DataSync · SFTP/FTPS 엔드포인트 제공 → Transfer Family
Redshift 외부 데이터	Spectrum	Federated Query	S3(Glue Catalog) → Spectrum · RDS/Aurora 라이브 → Federated Query
Redshift 공유	Data Sharing	UNLOAD 복사	라이브·복제 없음 → Data Sharing · 스냅샷 추출 → UNLOAD
정렬 키	Compound	Interleaved	프리픽스 순서 필터 → Compound · 다중 컬럼 동등(비권장) → Interleaved
DynamoDB 인덱스	GSI	LSI	다른 파티션 키·나중에 추가 → GSI · 같은 파티션·강한 일관성·생성 시 → LSI
DynamoDB 용량	On-demand	Provisioned	예측 불가·스파이크 → On-demand · 꾸준·비용 최적 → Provisioned + Auto Scaling
품질 도구	Glue Data Quality	DataBrew	파이프라인 규칙·게이트 → Glue DQ · 시각적 프로파일링·정제 → DataBrew
PII	Macie	Glue 민감 데이터 탐지	S3 저장 데이터 스캔·발견 → Macie · ETL 흐름 중 탐지·마스킹 → Glue
시크릿	Secrets Manager	SSM Parameter Store	자동 회전 필요 → Secrets Manager · 단순 설정값·무료 → SSM
VPC 엔드포인트	Gateway	Interface	S3·DynamoDB·무료 → Gateway · 그 외 서비스·PrivateLink → Interface

상황	A	B	가르는 기준
S3 암호화	SSE-S3	SSE-KMS	단순·기본 → SSE-S3 · 감사·회전·키 권한 분리 → SSE-KMS(+Bucket Key)
권한 세분화	IAM	Lake Formation	리소스·액션 수준 → IAM · 컬럼·행·셀·태그 기반 → Lake Formation
워크플로 종류	Standard	Express	장기·정확히 1회·감사 → Standard · 고빈도·≤5분·저비용 → Express



이 문서는 학습용 요약본이다. 서비스 스펙·한도·신규 기능은 AWS가 계속 갱신하므로, 시험 직전에는 공식 문서와 최신 Exam Guide를 확인하자. 오탈자·갱신 필요한 부분은 계속 고쳐 나간다.